

Chebyshev interpolation in Einstein–Boltzmann codes

Herman Sletmoen^{*}

Institute of Theoretical Astrophysics, University of Oslo, PO Box 1029 Blindern, 0315 Oslo, Norway

Received XX / Accepted XX

ABSTRACT

Einstein–Boltzmann codes compute theoretical predictions of cosmological models and rely heavily on interpolation in their independent variables: time τ , wavenumber k and multipole ℓ . We give a practical summary of interpolation with Chebyshev polynomials, which converges rapidly for smooth functions and thus pairs naturally with approximation-free Einstein–Boltzmann codes. By solving the perturbations and line-of-sight integrals at Chebyshev nodes in k and ℓ , we show that Chebyshev polynomials interpolate to higher precision than traditional cubic splines from fewer explicit solutions. On a set of example spectra for matter and the cosmic microwave background (CMB), we find up to four orders of magnitude lower interpolation error using the same number of points. For a typical CMB temperature spectrum computed with interpolation in both k and ℓ , Chebyshev polynomials converge to 10^{-4} – 10^{-5} relative error with only 50–80 points per variable, while cubic splines approach 10^{-4} error with 200 points, translating to a $2.5\times$ – $4\times$ speedup. The exact improvement depends on the target function and is generally more dramatic at high precision levels. Standard Chebyshev ℓ -interpolation needs line-of-sight integrals generalized to non-integer ℓ , but we show a way to avoid this by rounding the nodes to integers. Chebyshev interpolation is implemented in SymBoltz, which is available at <https://github.com/hersle/SymBoltz.jl>.

Key words. methods: numerical – cosmology: theory

1. Introduction

Einstein–Boltzmann codes, such as SymBoltz (Sletmoen 2026), CLASS (Blas et al. 2011) and CAMB (Lewis et al. 2000), are fundamental tools for computing theoretical predictions from cosmological models. They solve large and stiff ordinary differential equations (ODEs) and many line-of-sight integrals of the background and perturbation equations. This outputs source functions $S(\tau)$ and $S(\tau, k)$ and transfer functions $\Delta_\ell(k)$ of conformal time τ , perturbation wavenumber k and angular multipole ℓ .

Interpolation is one of the most effective optimizations in these codes. Instead of evolving every k and ℓ explicitly, they do this only on coarse grids and later interpolate to more values. This works because the output is smooth and saves time, but introduces interpolation errors. Codes have default settings tuned to balance performance and precision, but they slow down significantly with finer precision parameters that sample more points. In the literature, interpolation in parameter space with polynomials or neural network “emulators” has received much attention (e.g. Fendt & Wandelt 2007; Albers et al. 2019; Mancini et al. 2022; Bonici et al. 2024), while comparatively little attention has been paid to interpolation in the independent variables.

Since the pioneering COSMICS code (Ma & Bertschinger 1995), interpolation has been done with cubic splines. While solid for general-purpose interpolation, they are outperformed by interpolation with Chebyshev polynomials (Trefethen 2019). Cubic splines interpolate a function through N points with $n = N - 1$ piecewise cubic polynomials and an error that falls algebraically as n^{-4} . In contrast, Chebyshev interpolation fits a single n -th order polynomial through all points with an error that may drop exponentially as ρ^{-n} for some ρ . This interpolates to higher precision from fewer samples. It is also simple to implement with the barycentric formula (Berrut & Trefethen 2004), and un-

like equispaced polynomial interpolation it is stable (Trefethen 2011). However, the remarkably fast convergence depends on the function being smooth, as is often true in physics.

Traditional Einstein–Boltzmann codes such as CAMB and CLASS use approximation schemes to switch between different equations at different τ , k and ℓ , particularly in the perturbations (Blas et al. 2011, notably Figure 10). This is faster, but complicates both the physics and numerics. Moreover, it breaks smoothness by introducing kinks at the switches. Cubic splines are good for interpolating such functions because kinks affect their piecewise polynomials only locally. Chebyshev interpolation instead uses global polynomials, and a non-smooth feature at one point can slow convergence over the entire domain.

Recently, approximation-free Einstein–Boltzmann codes such as SymBoltz (Sletmoen 2026), DISCO-EB (Hahn et al. 2024) and PyCosmo (Refregier et al. 2018) have emerged. They integrate the same equations at all times to obtain fully smooth solutions, but spend more effort per point. In this paper, we show that approximation-free codes can leverage this smoothness with the rapid convergence of Chebyshev interpolation for performance and precision. We implement this feature in SymBoltz¹.

2. Computations in Einstein–Boltzmann codes

The equations in Einstein–Boltzmann codes are solved in stages for τ , k and ℓ , each interpolating solutions of earlier stages. Background functions $S(\tau)$ are interpolated in τ ; perturbation sources $S(\tau, k)$ in τ and k ; and angular spectra C_ℓ in ℓ (e.g. Callin 2006).

First, background and thermodynamics equations are solved for variables $S(\tau)$ that depend only on τ . For example, the Friedmann, continuity and Peebles equations are solved for the Hubble function $\mathcal{H}(\tau)$, free electron fraction $X_e(\tau)$, optical depth $\kappa(\tau)$, visibility function $v(\tau)$ and lookback time $\chi(\tau) = \tau_0 - \tau$.

^{*} Email: herman.sletmoen@astro.uio.no

¹ <https://hersle.github.io/SymBoltz.jl>

Second, large and stiff perturbation ODEs are integrated in τ for independent k -modes. This must interpolate the background solution in τ . It produces source functions $S(\tau, k)$ for CMB temperature (T), polarization (E), lensing (L) and matter overdensity (M ; see fig. 1 and Sletmoen (2026) for full variable definitions):

$$S^T = v \left(\frac{\delta_\gamma}{4} + \Psi + \frac{\Pi_\gamma}{16} \right) + e^{-\kappa} (\Psi + \Phi)' + (vu_b)' + \frac{3(v\Pi_\gamma)''}{16k^2}, \quad (1a)$$

$$S^E = \frac{3v\Pi_\gamma}{16(k\chi)^2}, \quad (1b)$$

$$S^L = (\Psi + \Phi) \frac{\chi_{\text{rec}}}{(\chi - \chi_{\text{rec}})\chi}, \quad (1c)$$

$$S^M = \delta_m + 3(1 + w_m) \frac{\mathcal{H}}{k^2} \theta_m. \quad (1d)$$

Third, line-of-sight integrals (Seljak & Zaldarriaga 1996)

$$\Delta_\ell^A(k) = \int_{\tau_i}^{\tau_0} d\tau S^A(\tau, k) j_\ell(k\chi) \quad (2)$$

are evaluated for a given source $S^A(\tau, k)$, which must be interpolated in τ and k . In turn, this is used to compute angular spectra

$$C_\ell^{AB} \propto \int_{k_{\min}}^{k_{\max}} dk k^2 P_0(k) \Delta_\ell^A(k) \Delta_\ell^B(k) \quad (\text{up to an } \ell\text{-scaling}). \quad (3)$$

Most $S(\tau, k)$ are smooth in k . But the spherical Bessel function $j_\ell(k\chi)$ makes $\Delta_\ell(k)$ oscillate rapidly in k , so the integrand of C_ℓ must be sampled at many k . A standard trick is to solve perturbations explicitly for only 10^2 – 10^3 k -modes, and then interpolate $S(\tau, k)$ to 10^3 – 10^4 values before line-of-sight integration. This is $10\times$ – $100\times$ faster than evolving the perturbations for all k .

Most C_ℓ are also smooth in ℓ . Similarly, it is faster to interpolate C_ℓ from some ℓ to every integer. This holds particularly in curved universes, which need hyperspherical Bessel functions.

3. Chebyshev polynomial interpolation

Any function $f(x)$ is interpolated through N points (x_i, f_i) by a polynomial $p_n(x)$ of order $n = N - 1$:

$$f(x) \approx p_n(x) = \sum_i f_i L_i(x) \quad \text{with} \quad L_i(x) = \frac{\prod_{j \neq i} (x - x_j)}{\prod_{j \neq i} (x_i - x_j)}. \quad (4)$$

The Lagrange polynomials satisfy $L_i(x_j) = \delta_{ij}$, so $p_n(x_i) = f_i$. We work with the standard domain $x \in [-1, 1]$, and map interpolation over arbitrary $y \in [a, b]$ with $x(y) = (2y - b - a)/(b - a)$.

The polynomial interpolation error is bounded by

$$|f(x) - p_n(x)| \leq \frac{M_{n+1}}{(n+1)!} |\omega(x)|, \quad \omega(x) = \prod_i (x - x_i), \quad (5)$$

where $M_n = \max |f^{(n)}(x)|$ on $x \in [-1, 1]$. Thus, the points should be placed in a way that minimizes the nodal polynomial $|\omega(x)|$.

Unintuitively, it is a bad idea to use uniformly spaced x_i . Then $\omega(x)$ decays very slowly with n near the edges and is easily outrun by $M_{n+1}/(n+1)!$ as $n \rightarrow \infty$. This is called Runge’s phenomenon and causes violent oscillations demonstrated in fig. 2a.

Two particularly excellent ways to place the points are at the Chebyshev nodes of the first or second kind (Trefethen 2019):

$$x_i^{(1)} = \cos \left(\frac{2i+1}{2n+2} \pi \right) \quad (i = 0, 1, \dots, n), \quad (6a)$$

$$x_i^{(2)} = \cos \left(\frac{i}{n} \pi \right) \quad (i = 0, 1, \dots, n). \quad (6b)$$

The points $x_i^{(1)}$ give the best possible bound $|\omega(x)| \leq 2^{-n}$ for all x . Meanwhile, $x_i^{(2)}$ bound $|\omega(x)| \leq 2^{-n+1}$ but include the endpoints $x = \pm 1$, which is sometimes more or less practical. In practice, $x_i^{(1)}$ and $x_i^{(2)}$ and any other set of points distributed with a similar $(1 - x^2)^{-1/2}$ -proportional density are effective. Here we use $x_i^{(2)}$.

The most attractive property of Chebyshev interpolation is that it converges faster the smoother $f(x)$ is (Trefethen 2011):

$$|f(x) - p_n(x)| \leq C \rho^{-n} \quad \text{if } f(x) \text{ is analytic}, \quad (7a)$$

$$|f(x) - p_n(x)| \leq C n^{-\nu} \quad \text{if } f(x) \text{ is differentiable } \nu \text{ times}. \quad (7b)$$

This is called “geometric” and “algebraic” convergence, respectively. Here C and $\rho > 1$ are constants that depend on $f(x)$.

Evaluating $p_n(x)$ is simple, fast and robust through the barycentric interpolation formula (Berrut & Trefethen 2004):

$$p_n(x) = \sum_i \frac{w_i}{x - x_i} f_i \bigg/ \sum_i \frac{w_i}{x - x_i}. \quad (8)$$

This involves the point-dependent interpolation weights

$$w_i = \frac{1}{\omega'(x_i)} = \frac{1}{\prod_{j \neq i} (x_i - x_j)}. \quad (9)$$

They can be precomputed for a general scheme, but for the Chebyshev nodes (6) one can instead just use the exact weights

$$w_i^{(1)} = (-1)^i \sin \left(\frac{(2i+1)}{2n+2} \pi \right), \quad (10a)$$

$$w_i^{(2)} = \begin{cases} (-1)^i/2 & \text{if } i = 0 \text{ or } i = n, \\ (-1)^i & \text{otherwise.} \end{cases} \quad (10b)$$

Barycentric interpolation is robust and uses only $O(n)$ operations per x . Despite suspicious divisions by $x - x_i$, it is stable in floating point arithmetic even as $x \rightarrow x_i$, and needs only a simple exception to return f_i if $x = x_i$ exactly (Higham 2004). In contrast, the Lagrange form (4) is unstable and uses $O(n^2)$ operations per x .

Our implementation creates a `ChebyshevInterpolator` from an interval $[a, b]$ and polynomial degree n , computes and transforms the nodes (6) to $[a, b]$, and then evaluates the barycentric formula (8) with weights (10) and any function values $f(x_i)$.

An alternative implementation expands $p_n(x) = \sum_m c_m T_m(x)$ in a series of Chebyshev polynomials $T_m(x)$. A Discrete Cosine Transform converts between the coefficients c_m and function values f_i . This lets us inspect the convergence of $|c_m|$ with m , and thus convergence of the full polynomial $p_n(x)$. It is not necessary, but is useful for error analysis and described in appendix A.

In contrast, piecewise interpolation methods join several low-order polynomials and converge more slowly. Cubic splines are popular for smooth data and interpolate with an error that falls as n^{-4} . They can struggle near the ends, where splines assume boundary conditions that need not match the data. In return, splines have better stability on arbitrary x -grids and respond locally to changes in the data. Splines are also faster to evaluate, but this is irrelevant when the cost of evaluating $f(x)$ dominates.

Thus, Chebyshev interpolation is very attractive when:

- $f(x)$ is smooth, to get the rapid convergence (7);
- $f(x)$ is queryable, to sample at the needed points (6);
- $f(x)$ is expensive, to benefit most from fewer samples.

These conditions are seldom met by observed data. But they often hold in theoretical applications such as Einstein–Boltzmann codes, which model smooth physics with demanding computations and can choose sampling points freely. In the next sections, we systematically consider interpolation in each independent variable: time τ , wavenumber k and multipole ℓ .

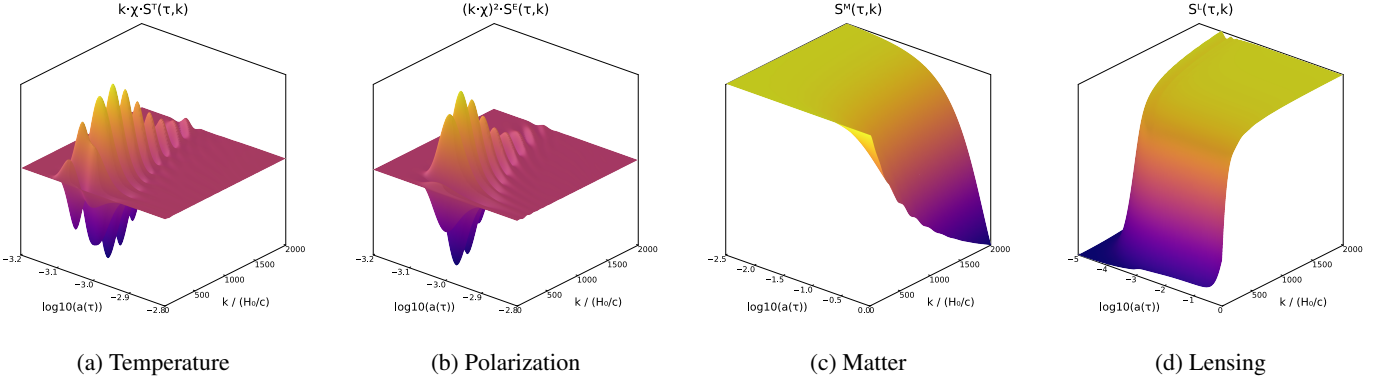


Fig. 1: Examples of the source functions (1) for temperature, polarization, matter and lensing (up to some normalization). The first two are shown around recombination, and the last two throughout cosmic time. The vertical scale is linear in arbitrary units.

4. Interpolation in time

We do *not* use Chebyshev interpolation in τ . Nevertheless, we find it instructive to discuss this aspect in order to understand the tradeoffs compared to Chebyshev interpolation in k and ℓ .

The background and perturbations for each k are ODEs $u'(\tau) = f(u, \tau)$ in τ . They have inherently local features, as the universe undergoes eras dominated by radiation, matter and dark energy, and shorter events such as recombination (see fig. 1a) and reionization. The ODEs are solved by adaptive methods that automatically adjust their time step to capture the local features. This conflicts with the rigid Chebyshev node placement (6).

Moreover, ODE solvers feature specialized dense output methods for interpolating over time steps. They are usually custom-tailored to the solver and optimized to maximize accuracy while reusing internal calculations of $f(u, \tau)$ at internal stages. For example, SymBoltz uses the implicit Rodas5P ODE solver by default, which has a specialized fourth-order interpolant (Steinebach 2023). Even without specialized dense output, the time derivative $u'(\tau) = f(u, \tau)$ can always be computed analytically from the ODE definition and solution $u(\tau)$ “for free”. One can then fall back to cubic Hermite interpolation, which takes both $u(\tau)$ and $u'(\tau)$ into account for improved accuracy. We do not use Chebyshev interpolation to compete with an interpolation method that is intrinsic to this algorithm.

To go beyond the scope of this paper and take Chebyshev polynomials in time seriously, one could use spectral ODE methods. Instead of evolving $u(\tau)$, they expand $u(\tau) = \sum_m a_m T_m(x(\tau))$ in Chebyshev polynomials $T_m(x(\tau))$ and derive equations for a_m from the ODE (e.g. Choustikov et al. 2023; Rana 2025).

5. Interpolation in wavenumber

In k , however, Chebyshev interpolation is a good candidate. The perturbation ODEs are independent for different k , contain no analytical k -derivatives for Hermite interpolation, and have a global nature in k as all scales inherit the background at a time τ .

Chebyshev k -interpolation consists of evolving the perturbation ODEs for the Chebyshev k -nodes (6) of a wavenumber interval $[k_{\min}, k_{\max}]$. Afterwards, any source $S(\tau, k)$ is computed at any k by doing barycentric interpolation in k over fixed τ -slices.

Figure 1 shows the four test source functions (1) that we interpolate. We interpolate $k\chi S^T(\tau, k)$ and $(k\chi)^2 S^E(\tau, k)$, as they are smoother and more similar in magnitude over k than $S(\tau, k)$.

Figure 2 shows the temperature source interpolated in k at recombination. It demonstrates that Chebyshev interpolation and equispaced cubic splines are stable, while Runge’s phenomenon

destabilizes equispaced polynomial interpolation. At $N \gtrsim 40$, the Chebyshev interpolation error falls rapidly below the cubic spline error. At $N = 80$, Chebyshev interpolation has fully converged to the numerical noise floor and is indistinguishable from solving all perturbation k -modes explicitly, while cubic splines have 4 orders of magnitude higher error. At $N = 400$, cubic splines have still not fully converged. This is the fast convergence (7) of Chebyshev interpolation for smooth functions.

Figure 3 generalizes the comparison to all four sources (1) and the full (τ, k) -space for $N = 100$. For the temperature and polarization sources, Chebyshev interpolation is 4 orders of magnitude more accurate than uniform cubic splines in a band around recombination at $a(\tau) \approx 10^{-3}$, where they are most important to capture. For the matter and lensing sources, Chebyshev interpolation is also more accurate and distributes the error uniformly over k . Note that cubic splines struggle particularly at the boundaries of the interpolation interval. This is because their natural boundary conditions assume that second derivatives are zero there, which disagrees with the underlying function.

Figure 3 also shows cubic splines interpolated on the Chebyshev grid (6). This only gives a lower error near the boundaries, confirming that the advantage of Chebyshev interpolation over uniform cubic splines is not merely due to its sample locations. In addition, the fourth row shows cubic splines with a non-uniform k -grid using CLASS’ default strategy, where k -modes are sampled with $25\times$ higher density for scales above the sound horizon at recombination than below it.² This much denser sampling gives cubic splines the lowest error for $k \lesssim 200 H_0/c$. Meanwhile, Chebyshev interpolation gives low average error throughout the domain with straightforward sampling.

Figure 4 compresses the error over (τ, k) to one root-mean-square number, and shows its convergence with more samples. The ranking of the methods depends on the source, and also crosses over with N . By this metric, Chebyshev interpolation achieves the lowest error for every source and $N \gtrsim 48$. This further supports that it is a better overall interpolation method when measured over the entire k -domain.

Figure 5 studies the decaying coefficients c_m in the Chebyshev series expansion $p_n(x) = \sum_m c_m T_m(x)$ of the interpolant. As appendix A explains, this is an equivalent Chebyshev interpolation method that offers a useful diagnostic: convergence of $|c_m|$ indicates convergence of $p_n(x)$. It has no direct counterpart with splines. The convergence undergoes three phases (here for S^T):

² Using `k_step_sub=0.05` and `k_step_super=0.002` in CLASS v3.3.4, but downsampling the grid from around 500 to 100 k -points.

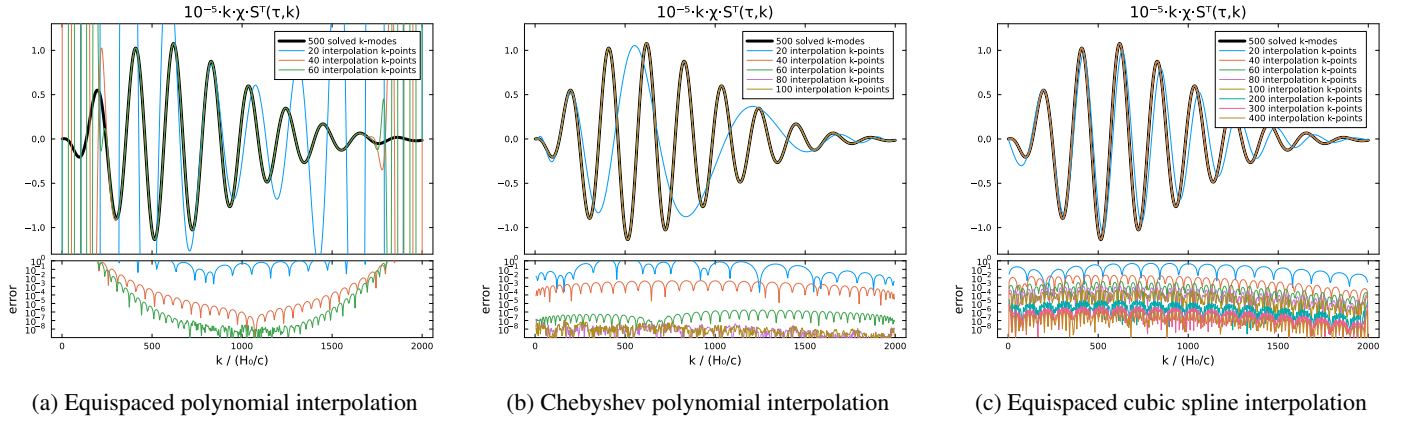


Fig. 2: Temperature source in fig. 1a at recombination interpolated with (a) equispaced polynomials, (b) Chebyshev polynomials and (c) equispaced cubic splines. The interpolated values are compared to explicit solutions for 500 perturbation k -modes.

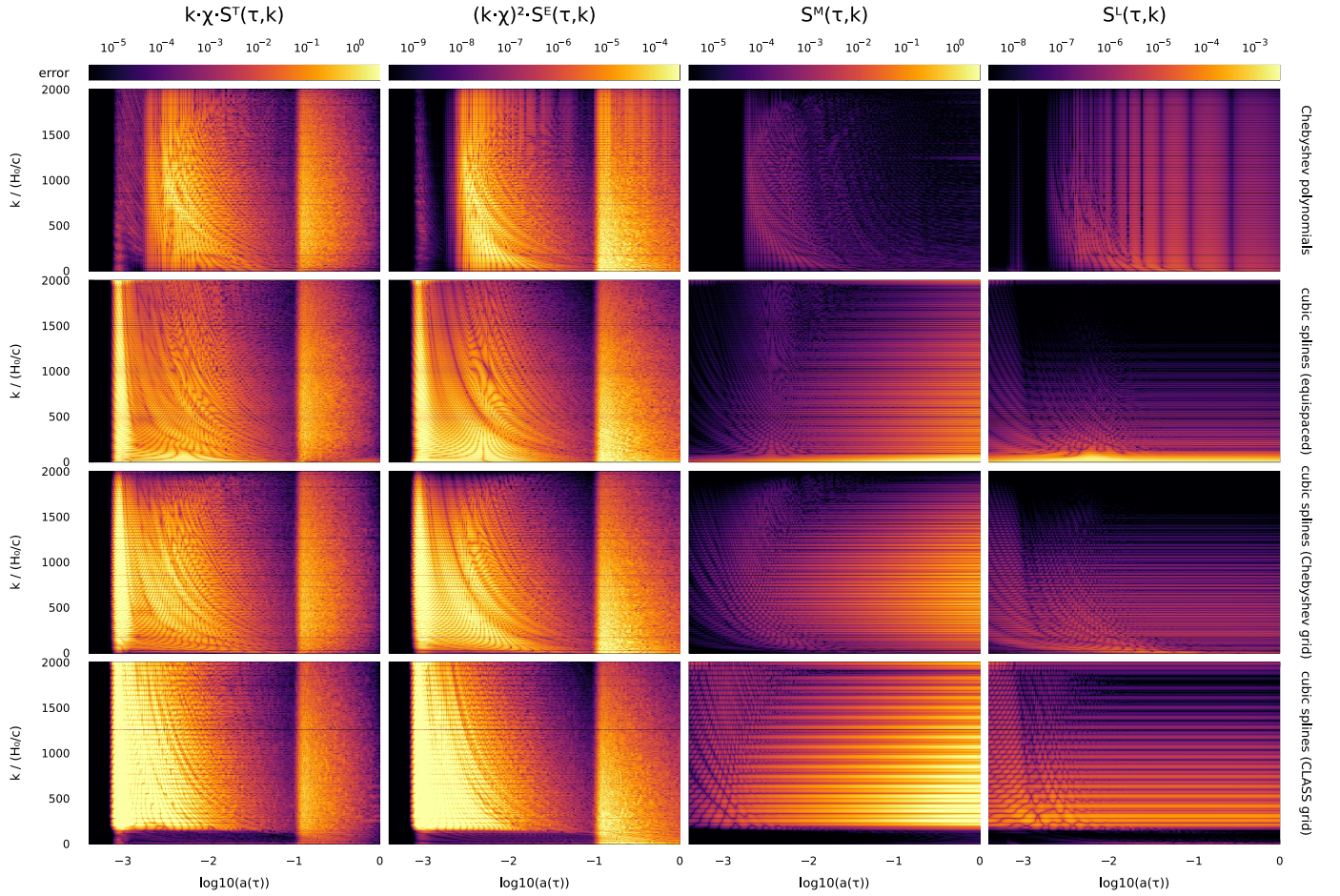


Fig. 3: Interpolation error of temperature, polarization, matter and lensing source functions in fig. 1 (left to right) interpolated in k from 100 points using Chebyshev polynomials, equispaced cubic splines, cubic splines on a Chebyshev grid and cubic splines on a CLASS-like grid (top to bottom). As in fig. 2, the error is computed relative to 500 explicitly solved perturbation k -modes.

- For $0 \lesssim m \lesssim 25$, the interpolant is still resolving the features of the source function and $|c_m|$ has not yet begun to converge.
- For $25 \lesssim m \lesssim 75$, the interpolant has captured the main features and $|c_m|$ decays rapidly. The steepest fall is a line in the log-linear plot, indicating geometric convergence (7a).
- For $m \gtrsim 75$, the interpolant and $|c_m|$ fully converge to the numerical noise floor. Smaller ODE solver tolerances lower

the precision floor. This matches the convergence with 60–80 points in fig. 2b, and there is little gain in using more points.

In principle, the code could use this decay to choose n adaptively.

Thus, we find Chebyshev interpolation in k to be a better overall method for the whole k -domain. At high precision levels, it interpolates with orders of magnitude lower error than cubic splines from fewer explicit solutions to the perturbation ODEs.

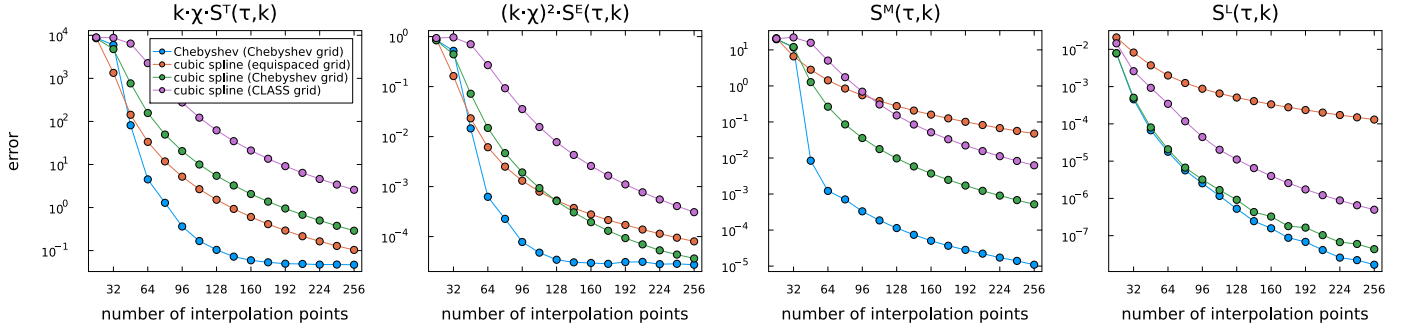


Fig. 4: Root-mean-square of the interpolation error over (τ, k) using k -interpolation with Chebyshev polynomials, uniform cubic splines, cubic splines with Chebyshev points and cubic splines with CLASS-like k -sampling (as in fig. 3) as the number of samples increases.

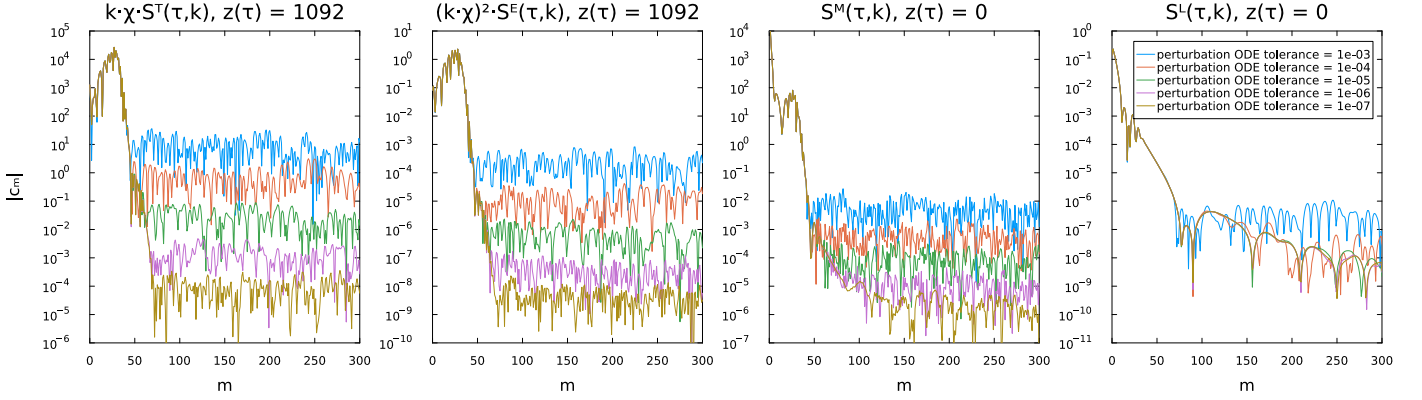


Fig. 5: Coefficient magnitudes $|c_m|$ in the Chebyshev polynomial expansion $S(\tau, k) \approx \sum_{m=0}^{300} c_m(\tau) T_m(x(k))$ of the source functions in fig. 1. The temperature and polarization sources are evaluated at recombination, and the matter and lensing sources today. The noise floor for the interpolation error is lowered by stricter ODE solver tolerances for the perturbations.

6. Interpolation in multipole

Angular power spectra C_ℓ are also smooth functions of the multipole ℓ . Instead of computing C_ℓ explicitly with line-of-sight integration for every ℓ , Einstein–Boltzmann codes do this only for some ℓ and then interpolate to all desired integer ℓ . This again makes Chebyshev interpolation a good candidate.

However, the discrete nature of ℓ raises a difficulty. As C_ℓ is physically interpreted only at integer ℓ , other codes interpolate from an integer ℓ -grid (e.g. using every low ℓ and every 30th high ℓ). This works with cubic splines because their stability is better for arbitrary grid configurations. But it is incompatible with standard Chebyshev interpolation, because the node placement (6) forces the interpolation grid to both integer and non-integer ℓ . This was not a concern in section 5, where sources vary continuously with k . We show two different ways to solve this problem.

The first way is to proceed with standard Chebyshev interpolation, but generalize the line-of-sight integrals (2) and spherical Bessel functions $j_\ell(x)$ to arbitrary (integer and non-integer) ℓ . This is unproblematic from a physical point of view, as $j_\ell(x)$ is the solution that is regular at $x = 0$ of the ODE

$$x^2 \frac{d^2 j_\ell}{dx^2} + 2x \frac{dj_\ell}{dx} + (x^2 - \ell(\ell + 1)) j_\ell = 0. \quad (11)$$

This is continuous and smooth in both ℓ and x , so interpolation from non-integer to integer ℓ converges. The important part is not the precise value of $j_\ell(x)$ for non-integer ℓ , but that there is

a continuation of $j_\ell(x)$ between integer ℓ that is smooth.³ Cubic spline interpolation between integer-only ℓ also relies on this property. When a standard `ChebyshevInterpolator` is used to interpolate over $\ell \in [\ell_{\min}, \ell_{\max}]$, `SymBoltz` follows this path by evaluating $j_\ell(x)$ at arbitrary ℓ with the `Bessels.jl` package.

The second way is to compute $j_\ell(x)$ only at integer ℓ , but round Chebyshev nodes to their nearest integers. These non-standard points break the Chebyshev weights (10) and coefficients (A.10), but barycentric interpolation is still straightforward using the general weights (9). This near-Chebyshev method is robust, as polynomial interpolation is stable for any points with density similar to the Chebyshev nodes (6). It also helps that C_ℓ is needed for 1000s of multipoles, which mitigates discreteness effects when rounding 10s–100s of interpolation points. `SymBoltz` implements a `ChebyshevIntegerInterpolator` with this strategy, and errors if the requested order is so high that multiple nodes round to the same ℓ .

Figure 6 compares ℓ -interpolation performance of cubic splines versus Chebyshev polynomials. To fix the computational cost, we use 70 ℓ -points with each method. For each method, we also require that the same ℓ -grid interpolates all three spectra C_ℓ^{TT} , C_ℓ^{EE} and C_ℓ^{LL} , so the precomputed $j_\ell(x)$ is common.

For C_ℓ^{TT} and C_ℓ^{EE} , Chebyshev interpolation converges to 3–4 orders of magnitude smaller error for $\ell \gtrsim 100$ than cubic splines. But for C_ℓ^{LL} , the cubic spline does better than a single Chebyshev

³ Extending $j_\ell(x)$ to continuous ℓ is analogous to analytically continuing the factorial $n!$ to the gamma function $\Gamma(n + 1)$, for example.

⁴ <https://github.com/JuliaMath/Bessels.jl>

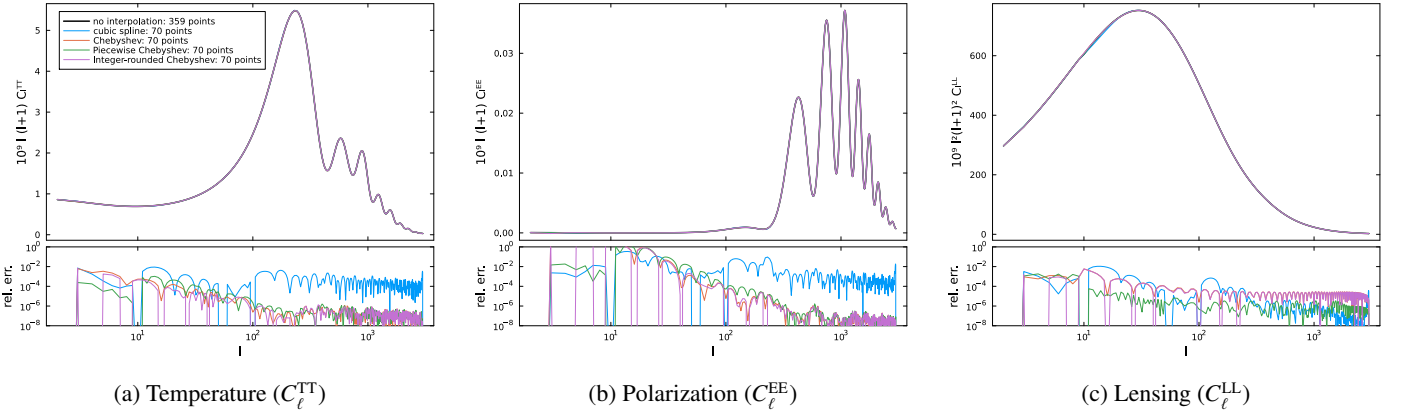


Fig. 6: CMB temperature, polarization and lensing spectra interpolated in ℓ with cubic spline versus Chebyshev polynomials. We interpolate $\ell^5 C_\ell$ as a function of ℓ , which made all methods perform best. Cubic splines use 8, 9 and 55 samples on $\ell \in [2, 10]$, $[10, 100]$ and $[100, 3000]$, respectively. Standard Chebyshev interpolation fits a single 69th order polynomial through the Chebyshev nodes on $\ell \in [2, 3000]$. The piecewise variant joins a 5th order polynomial on $\ell \in [2, 10]$ with a 64th order polynomial on $\ell \in [10, 3000]$. The integer-rounded variant instead runs the degree-69 polynomial through the integers that are closest to the standard Chebyshev nodes. Every interpolated spectrum is compared to explicit line-of-sight integration for 359 ℓ -values.

polynomial. This is because the Limber approximation switches on for $\ell \geq 10$ and makes a kink in C_ℓ , leading to only algebraic n^{-1} -convergence (7b). The cubic spline then wins with n^{-4} convergence because it responds locally, so high ℓ is unaffected by the kink. To fix this, we make a piecewise Chebyshev interpolant with two independent polynomials for $\ell \in [2, 10]$ and $\ell \in [10, 3000]$ (Limber approximation off/on). This lowers the Chebyshev interpolation error to the level of cubic splines.

Chebyshev interpolation is also simpler. Fixing the computational cost to exactly 70 ℓ -points, we first tried a cubic spline with a uniform ℓ -grid, but this led to large interpolation errors for small ℓ . After progressively refining the ℓ -grid with trial and error, we ended up with a high-medium-low density ℓ -grid over the subintervals of $\ell \in [2, 10, 100, 3000]$. Meanwhile, Chebyshev interpolation performs better with a straightforward ℓ -grid.

The figure also shows that integer-rounded and standard Chebyshev ℓ -interpolation perform equally well.

Here we concentrated on a flat cosmology with spherical Bessel functions. In curved universes they generalize to hyperspherical Bessel functions (e.g. Lesgourgues & Tram 2014). They are harder to compute, typically using recurrence relations between integer ℓ (Tram 2017) instead of solving the ODE (11), which receives an extra curvature-dependent term. We therefore anticipate that the integer-rounded Chebyshev interpolation strategy would be easiest to generalize to curved geometries.

Thus, Chebyshev interpolation is better also in ℓ . It reaches up to several orders of magnitude lower error in CMB spectra than cubic splines with the same number of samples. The emulator Capse.jl for $C_\ell(p)$ of cosmological parameters p also uses it, but their main purpose is to train on fewer independent coefficients $a_m(p)$ in the Chebyshev series $C_\ell(p) = \sum_m a_m(p) T_m(x(\ell))$ instead of every $C_\ell(p)$ (Bonici et al. 2024). Appendix A links the coefficients to our barycentric approach.

7. Interpolation in wavenumber and multipole

In the previous section on ℓ -interpolation, we always used Chebyshev k -interpolation of the source functions $S(\tau, k)$ before line-of-sight integration in order to isolate the effect to ℓ .

Figure 7 instead shows the combined effect: the CMB temperature spectrum C_ℓ^{TT} interpolated with cubic splines in both k

and ℓ , versus Chebyshev interpolation in both k and ℓ . Here we skip the cosmic variance-dominated $\ell \lesssim 50$ in fig. 6a, and take the root-mean-square of the relative C_ℓ error for the smoothest $50 \leq \ell \leq 2500$ as a function of the number of k -points and ℓ -points. Chebyshev interpolation converges to the 10^{-4} – 10^{-5} -level with only 50–80 points in both k and ℓ , while cubic splines approach the 10^{-4} -level around 200 points. This speeds up the perturbations and hence the full computation by $2.5 \times$ – $4 \times$ for a fixed high precision level (e.g. 0.2 s for 50×50 points vs 0.8 s for 200×200 on our machine). Line-of-sight integration is sped up by a similar factor, but is not the main bottleneck of the computation in flat universes. In curved universes, hyperspherical Bessel functions are more expensive to compute, so the speedup of line-of-sight integration may become more visible there.

8. Conclusion

We implemented interpolation with Chebyshev polynomials with the barycentric formula in the approximation-free Einstein–Boltzmann code SymBoltz. By solving the perturbation ODEs and line-of-sight integrals at Chebyshev nodes in k and ℓ , we reached significantly lower interpolation error than traditional cubic splines using the same number of samples.⁵

The rapid convergence of Chebyshev interpolation for smooth functions forms a synergy with approximation-free Einstein–Boltzmann codes. In contrast, approximation-based codes switch equations in τ , k and ℓ to solve each point faster, but complicate the physics and numerics and break smoothness. In other words, Chebyshev interpolation provides approximation-free codes a unique way to win back performance and precision.

Standard Chebyshev interpolation in ℓ requires line-of-sight integrals and spherical Bessel functions generalized to non-integer ℓ , but we showed how to avoid this by rounding Chebyshev nodes to their nearest integer. This near-Chebyshev strategy performs comparably and would be easier to use in curved universes, which need hyperspherical Bessel functions that are typically computed with integer-based recurrences and polynomials.

⁵ Results in this paper were generated with SymBoltz 1.7.0 and the script <https://github.com/hersle/SymBoltz.jl/blob/v1.7.0/scripts/chebyshev.jl>.

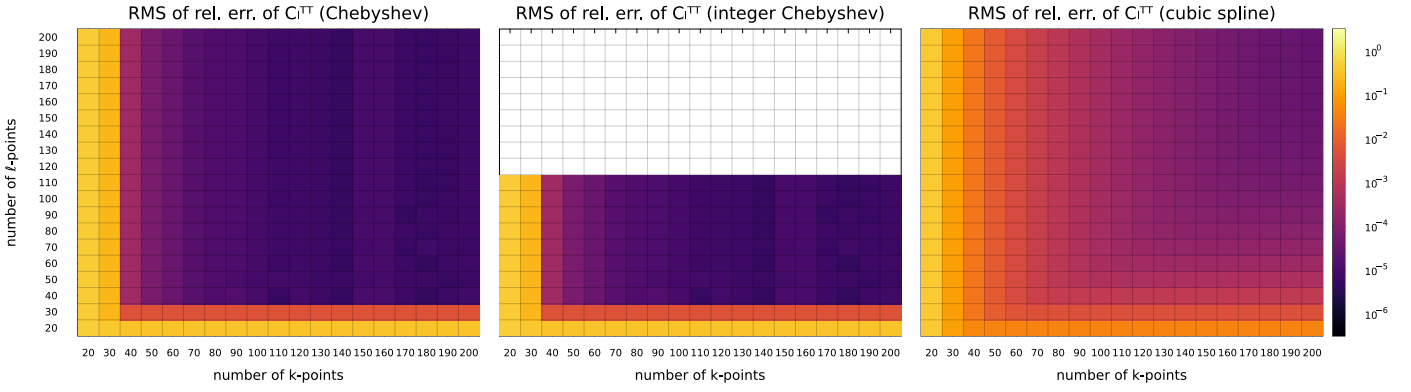


Fig. 7: Root-mean-square of the relative error ($(\sum_{\ell}(C_{\ell}^{\text{TT}}/\bar{C}_{\ell}^{\text{TT}} - 1)^2/N_{\ell})^{1/2}$) of the CMB temperature power spectrum C_{ℓ}^{TT} for $50 \leq \ell \leq 2500$, computed with Chebyshev interpolation in both k and ℓ versus equispaced cubic splines in both k and ℓ with different numbers of samples. The error is relative to a reference spectrum $\bar{C}_{\ell}^{\text{TT}}$ computed with Chebyshev interpolation from 1025 k -points and 513 ℓ -points. The middle plot uses integer-rounded Chebyshev ℓ -interpolation and is capped at 110 Chebyshev nodes, because a higher number would round several ℓ -nodes to the same integer.

If implemented carefully, we believe approximation-based codes could also benefit from Chebyshev interpolation. As it is a global method, imperfections at one point can slow interpolation convergence over the whole domain. For example, we saw that the Limber approximation slowed convergence of Chebyshev ℓ -interpolation, but restored it by interpolating piecewise for $\ell < \ell_{\text{Limber}}$ and $\ell \geq \ell_{\text{Limber}}$. The same strategy could be used for approximations in k -space, but is not straightforward when the switching criterion varies with both τ and k . Cubic splines are safer with approximations because their error decays faster around local non-smooth features, but converge more slowly.

Interpolation is only one of many computational methods in Einstein–Boltzmann codes. Our scope here was to study convergence only of interpolation, while fixing other precision parameters. Of course, lower precision of line-of-sight quadrature, the Limber approximation and other methods may overshadow the high precision of Chebyshev interpolation. Convergence with respect to *all* precision parameters calls for more research into high-precision methods, such as better line-of-sight integration and beyond-Limber methods (e.g. [Schöneberg et al. 2018](#); [Fang et al. 2020](#); [Chiarenza et al. 2024](#); [Reischke 2025](#)).

We believe Chebyshev interpolation is a step forward for simpler Einstein–Boltzmann codes with numerical algorithms that make better use of the physical properties of the equations, instead of resorting to complicated approximation schemes.

Acknowledgements. I thank Hans A. Winther for providing valuable feedback on drafts of this paper. I thank Michael Helton for creating the Bessels.jl package, which made it easy to compute spherical Bessel functions with both integer and non-integer order in this paper. I thank Steven G. Johnson for creating the FastChebInterp.jl package, which made it easy to inspect the convergence of Chebyshev coefficients in this paper. This research was supported by the Research Council of Norway under project number 325113.

References

- Albers, J., Fidler, C., Lesgourgues, J., Schöneberg, N., & Torrado, J. 2019, *J. Cosmol. Astropart. Phys.*, 2019, 028
- Berrut, J.-P. & Trefethen, L. N. 2004, *SIAM Rev.*, 46, 501
- Blas, D., Lesgourgues, J., & Tram, T. 2011, *J. Cosmol. Astropart. Phys.*, 2011, 034
- Bonici, M., Bianchini, F., & Ruiz-Zapatero, J. 2024, *Open J. Astrophys.*, 7
- Callin, P. 2006, *arXiv e-prints*
- Chiarenza, S., Bonici, M., Percival, W., & White, M. 2024, *Open J. Astrophys.*, 7
- Choustikov, N., Vlah, Z., & Challinor, A. 2023, *Phys. Rev. D*, 108, 023529

- Clenshaw, C. W. 1955, *Math. Comput.*, 9, 118
- Fang, X., Krause, E., Eifler, T., & MacCrann, N. 2020, *J. Cosmol. Astropart. Phys.*, 2020, 010
- Fendt, W. A. & Wandelt, B. D. 2007, *Astrophys. J.*, 654, 2
- Hahn, O., List, F., & Porqueres, N. 2024, *J. Cosmol. Astropart. Phys.*, 06, 063
- Higham, N. J. 2004, *IMA J. Numer. Anal.*, 24, 547
- Lesgourgues, J. & Tram, T. 2014, *J. Cosmol. Astropart. Phys.*, 2014, 032
- Lewis, A., Challinor, A., & Lasenby, A. 2000, *Astrophys. J.*, 538, 473
- Ma, C.-P. & Bertschinger, E. 1995, *Astrophys. J.*, 455, 7
- Mancini, A. S., Piras, D., Alsing, J., Joachimi, B., & Hobson, M. P. 2022, *Mon. Not. R. Astron. Soc.*, 511, 1771
- Rana, A. 2025, *arXiv e-prints*
- Refregier, A., Gamper, L., Amara, A., & Heisenberg, L. 2018, *Astron. Comput.*, 25, 38
- Reischke, R. 2025, *J. Open Source Softw.*, 10, 8618
- Schöneberg, N., Simonović, M., Lesgourgues, J., & Zaldarriaga, M. 2018, *J. Cosmol. Astropart. Phys.*, 2018, 047
- Seljak, U. & Zaldarriaga, M. 1996, *Astrophys. J.*, 469, 437
- Sletmoen, H. 2026, *Astron. Astrophys.*, 707, A128
- Steinebach, G. 2023, *BIT Numer. Math.*, 63, 27
- Tram, T. 2017, *Commun. Comput. Phys.*, 22, 852
- Trefethen, L. N. 2011, *Maths. Today*, 47, 184
- Trefethen, L. N. 2019, *Approximation Theory and Approximation Practice*, Extended Edition (Society for Industrial and Applied Mathematics)

Appendix A: Chebyshev coefficients

Instead of interpolating with the barycentric formula in section 3, a function can be approximated with a series of Chebyshev polynomials. This alternative approach to Chebyshev interpolation is particularly useful for error analysis. We give a brief summary of this approach based on Trefethen (2019).

The Chebyshev polynomials of the first kind are defined as

$$T_n(x) = \cos(n \arccos x) \quad (x \in [-1, 1]). \quad (\text{A.1})$$

To see it is really a polynomial, evaluate $T_0(x)$ and $T_1(x)$ explicitly and use the identities $\cos(n\theta \pm \theta) = \cos(n\theta)\cos(\theta) \mp \sin(n\theta)\sin(\theta)$ with $\theta = \arccos x$ to get the polynomial recurrence

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x). \quad (\text{A.2})$$

The Chebyshev polynomials are an orthogonal basis for expanding a function $f(x)$ on $x \in [-1, 1]$ in a unique series

$$f(x) = \sum_{m=0}^{\infty} a_m T_m(x). \quad (\text{A.3})$$

They are orthogonal with respect to the inner product (change variables to $x = \cos \theta$ and use orthogonality of cosines)

$$\int_{-1}^1 \frac{T_m(x)T_{m'}(x)}{\sqrt{1-x^2}} dx = \begin{cases} 0 & \text{if } m \neq m', \\ \pi & \text{if } m = m' = 0, \\ \pi/2 & \text{if } m = m' \geq 1, \end{cases} \quad (\text{A.4})$$

so the coefficients that represent a given function $f(x)$ are

$$a_m = \begin{cases} \frac{1}{\pi} \int_{-1}^1 \frac{f(x)T_0(x)}{\sqrt{1-x^2}} dx & \text{if } m = 0, \\ \frac{2}{\pi} \int_{-1}^1 \frac{f(x)T_m(x)}{\sqrt{1-x^2}} dx & \text{if } m \geq 1. \end{cases} \quad (\text{A.5})$$

These coefficients represent $f(x)$ in the infinite-dimensional space (A.3), and are not very practical.

A.1. Chebyshev projection

One way to approximate $f(x)$ is to take the coefficients (A.5), but truncate the infinite series (A.3) at a finite n :

$$f(x) \approx \hat{p}_n(x) = \sum_{m=0}^n a_m T_m(x). \quad (\text{A.6})$$

This is called Chebyshev projection, because dropping higher-order coefficients effectively projects $f(x)$ from an infinite-dimensional space of continuous functions onto a finite lower-dimensional space of polynomials.

Importantly, this is not interpolation because it does not pass through a given sample of function values. Instead it deals with the integrals (A.5), which need $f(x)$ to be evaluated at arbitrary x . This can be appropriate when one has an expression for $f(x)$ and can evaluate the integrals analytically, but it is impractical for other purposes.

A.2. Chebyshev interpolation

A practical interpolation method is to construct the polynomial $p_n(x)$ that passes through $f(x)$ at certain points x_i . However, instead of expanding it in Lagrange polynomials as in section 3, we can expand it in a series of Chebyshev polynomials:

$$f(x) \approx p_n(x) = \sum_{m=0}^n c_m T_m(x) \quad \text{such that} \quad p_n(x_i) = f(x_i). \quad (\text{A.7})$$

It is convenient to select x_i as the extrema of $T_n(x)$. The trigonometric definition (A.1) reaches $|T_n(x)| = 1$ when $n\theta = i\pi$ for integer i , so its extrema are

$$x_i = \cos\left(\frac{i\pi}{n}\right) \quad (0 \leq i \leq n). \quad (\text{A.8})$$

Analogously to the continuous inner product (A.4), the extrema give rise to a discrete orthogonality relationship:

$$\sum_{i=0}^n{}'' T_m(x_i)T_{m'}(x_i) = \begin{cases} 0 & \text{if } m \neq m', \\ n & \text{if } m = m' \in \{0, n\}, \\ n/2 & \text{if } m = m' \in [1, n-1], \end{cases} \quad (\text{A.9})$$

where the double-primed sum halves its first and last terms. Thus, the coefficients c_m that interpolate a given $f(x)$ are

$$c_m = \begin{cases} \frac{1}{n} \sum_{i=0}^n{}'' f(x_i) \cos\left(\frac{im\pi}{n}\right) & \text{if } m \in \{0, n\}, \\ \frac{2}{n} \sum_{i=0}^n{}'' f(x_i) \cos\left(\frac{im\pi}{n}\right) & \text{if } m \in [1, n-1]. \end{cases} \quad (\text{A.10})$$

This is a type-I Discrete Cosine Transform (DCT). It can be computed with only $O(n \log n)$ operations. This special property makes Chebyshev interpolation particularly attractive.

Once c_m are known, the interpolating series (A.7) is easily evaluated for any x with $O(n)$ operations. Forward iteration through the recurrence (A.2) may be most natural, but this is unstable for $|x| \rightarrow 1$. A stable way is to iterate backwards with the Clenshaw (1955) algorithm with an intermediate variable $b_m(x)$:

$$b_{n+1}(x) = b_{n+2}(x) = 0, \quad (\text{A.11a})$$

$$b_m(x) = c_m + 2xb_{m+1}(x) - b_{m+2}(x) \quad (m = n, \dots, 0), \quad (\text{A.11b})$$

$$p_n(x) = \frac{1}{2}(c_0 + b_0(x) - b_2(x)). \quad (\text{A.11c})$$

Interpolating with the series (A.7) is equivalent to the barycentric formula (8) with the Chebyshev points (6b) and weights (10b) of the second kind, as the polynomial is of the same order and goes through the same points. The DCT (A.10) converts the function values $f(x_i)$ of the barycentric approach to the coefficients c_m of the series approach. While the barycentric approach is simpler, faster and more flexible, the advantage of the series is that the coefficients provide a useful diagnostic for the maximum interpolation error (Trefethen 2019, Chapter 4):

$$|f(x) - p_n(x)| \leq 2 \sum_{m=n+1}^{\infty} |a_m|. \quad (\text{A.12})$$

This bound involves the projection coefficients a_m , not the interpolation coefficients c_m . The two differ by aliasing effects that diminish as the series converges. Thus $c_m \rightarrow a_m$ in practice, so we can compute the DCT coefficients (A.10) and check for their convergence, which also implies convergence of the full $p_n(x)$.

SymBoltz always interpolates with the barycentric formula. In addition, we added a way to compute the DCT for the coefficients (A.10) with the FastChebInterp.jl⁶ package, as an additional way to inspect the interpolation convergence in fig. 5.

⁶ <https://github.com/JuliaMath/FastChebInterp.jl>